

Mikrodenetleyici Tabanlı Otonom Labirent Çözücü Robot ve Yörünge Optimizasyon Algoritması Tasarımı

Projenin Amacı

Bu laboratuvar çalışmasının amacı; temel "gördüğün çizgiye git" mantığındaki reaktif çizgi izleme algoritmalarını bir üst seviyeye taşıyarak sistemi bilişsel bir yapıya dönüştürmektir. Robotun karmaşık yol ağlarında (labirent) geometrik düğüm noktalarını (kavşakları) donanımsal olarak algılaması, önceden tanımlanmış topolojik kurallara (sezgisel algoritmalar) göre otonom yön kararları verebilmesi hedeflenmektedir. Projenin asıl mühendislik sınavı ise; geçilen rotayı mikrodenetleyicinin hafızasına (SRAM) alarak "çıkılmaz sokakları" bulmak, bu hatalı yolları veri dizisinden (array) matematiksel kurallarla elemek ve robotun hedefe giden en kısa/pürüzsüz yolu otonom olarak hesaplayabildiği kapalı çevrim bir navigasyon sistemi tasarlamaktır.

Kullanılacak Temel Donanımlar

- 1 adet Mikrodenetleyici Geliştirme Kartı (Arduino Nano / Uno vb.)
- Çoklu Optik Yansıma Sensör Dizilimi (Örn: En az 5 veya daha fazla kanallı geniş açılı IR sensör dizisi)
- 1 adet Çift Kanallı Motor Sürücü Entegresi veya Modülü (L298N, TB6612FNG vb.)
- 2 adet Diferansiyel Sürüşe Uygun Redüktörlü DC Motor ve Tekerlek Takımı
- Kararlılık İçin Güç İzolasyon ve Regülasyon Elemanları (Lojik besleme ile motor beslemesinin elektriksel olarak ayrıştırılması için)
- Şase, Sarhoş Teker (Caster Wheel) ve Taşınabilir Güç Kaynağı (Li-Po Batarya vb.)

Sistemin Çalışma Prensipleri

Sistem temelde iki fazlı bir asenkron durum makinesi (state machine) olarak çalışır: **Keşif Fazı** ve **Optimizasyon (Hızlı Çözüm) Fazı**. Robot, ön tarafında bulunan genişletilmiş sensör dizilimi sayesinde bulunduğu zeminin topolojisini dijital bir bit dizisi olarak okur. Keşif aşamasında sistem, karşısına çıkan tüm kavşaklarda sürekli olarak belirli bir sezgisel kuralı (Örneğin: "Daima Sol Duvarı Takip Et") uygular ve yaptığı her spesifik dönüşü mikrodenetleyicinin rastgele erişimli belleğine (SRAM) bir karakter veya sayı dizisi (array) elemanı olarak kaydeder. Hedefe ulaşıldığında keşif biter ve sistemin işlemcisi devreye girerek bellekteki bu devasa rotayı analiz eder. Yapılan gereksiz U-dönüşleri ve çıkılmaz sokak girişleri matematiksel vektör sadeleştirme kurallarıyla bellekten silinir. Robot tekrar başlangıç noktasına konulduğunda, artık sensörleriyle yolu okuyarak körleme gitmek yerine belleğindeki bu "Kusursuz Rotayı" referans alarak hedefe en kısa sürede, hiçbir çıkılmaz sokağa girmeden ulaşır.

1. Çoklu Sensör Topolojisi ve D ğ m Noktası (Kavşak) Karakterizasyonu

- **Lojik Durum (State) Analizi:** Standart bir çizgi izleyen robotta sistemi yolda tutmak i in 2 veya 3 sens r yeterliyken, labirent   z c  bir sistemde tam kavşak (Cross), T-kavşak, saęa/sola keskin d n ş veya  ıkmaz sokak (Dead-end) gibi durumları ayırt edebilmek i in sens r diziliminin her iki yana doęru geniřletilmiř ( rneęin 5'li veya 8'li bir dizilim) olması zorunludur.
- **Tasarım ve Haritalama G revi:** Her bir sens r n okuduęu zıtlık durumu (Siyah: 1, Beyaz: 0) bir araya geldięinde iřlemci i in 5-bitlik lojik bir kelime ( rn: 11011 veya 10001) oluřturacaktır. Karřılařılabilecek t m fiziksel yol geometrilerinin (Tam Kesiřim, Sadece Sola D n ř, Sadece Saęa D n ř,  ıkmaz Sokak, Sola D n ř ve İleri Yolun aynı anda a ık olması durumu vb.) bu bit diziliminde hangi spesifik mantıksal durumlara denk geldięini teorik olarak tanımlayan bir "Durum Karar Matrisi" (State Matrix) oluřturunuz.

2. Topolojik   z m Yaklařımı (Heuristik Y nlendirme Kuralı)

- **Algoritmik Kurgu Tespiti:** Hi bir  n bilgisi (haritası) olmayan bir labirentin i erisinde hedefe ulařmayı kesin olarak garanti eden temel sezgisel (heuristik) algoritmaların literat rdeki matematiksel ve mantıksal altyapısı arařtırılmalıdır.
- **Karar Hiyerarřisi Tasarımı:** Robot birden fazla se eneęin olduęu bir d ę m noktasına ( rneęin hem ileri gidilebilen hem de sola d n lebilen bir T-Kavşak) geldięinde, sistemin iřlem  ncelięi hangi y nde olmalıdır? Eęer "Sol El Kuralı" (Left-Hand Rule) uygulanacaksa; algoritmanın if/else veya switch/case yapısındaki kontrol sıralaması ( ncelik 1: Sola d n lebilir mi?,  ncelik 2: İleri gidilebilir mi?,  ncelik 3: Saęa d n lebilir mi? vb.) doęrudan donanımsal okumalara dayandırılarak koda nasıl entegre edilmelidir?

3. Hafıza Y netimi ve Y r nge Optimizasyonu (Sadeleřtirme Matematięi)

- **Problem Tespiti:** Keřif turu sırasında robot  ıkmaz sokaklara girdięinde, yolun bittięini ( rneęin t m sens rlerin beyaz okuduęu an) algılayacak ve geriye doęru kendi eksenini etrafında bir "U-D n ř " (Geri D n) yapacaktır. Belleęe sırasıyla kaydedilen bu eylemler ( rn: Sol, İleri, U-D n ř, Saę...) ham halindedir ve robotu tekrar aynı  ıkmaz sokaklara sokacaęı i in mutlak bir optimizasyon gerektirir.
- **Matematiksel D n ř m ve Filtreleme G revi:** Bellekte tutulan bu komut dizisinde, bir U-d n ř n n (Geri) etrafındaki komutlarla birleřtięinde fiziksel uzayda  ok daha basit tekil bir d n ře eřit olduęu g r lecektir.  rneęin; sistemin sırasıyla "Sola D n + U-D n ř  Yap + Sola D n" kaydı tuttuęu bir manevra, uzaysal olarak deęerlendirildięinde aslında labirentte hi  sapmadan doęrudan "D z Git" eylemi ile aynı konuma  ıkmaktadır.

- **Algoritma Tasarımı:** Dizideki bu "U-Dönüşü (U)" parametrelerini barındıran tüm olası üçlü kombinasyonları (Sağ+U+Sol, İleri+U+Sağ, Sol+U+İleri vb.) tek bir mantıksal eyleme indirgeyecek sadeleştirme kurallarını kağıt üzerinde vektörel olarak tek tek hesaplayınız. Yazılım algoritmanızda bu diziyi (array) sürekli tarayan, kurallara uyan kısımları diziden silip yerlerine eşdeğer optimize komutu yazan (ve diziyi kaydıran) "Veri Manipülasyon ve Optimizasyon Fonksiyonunu" tasarlayınız.

Proje İş Paketleri ve Tasarım Yol Haritası

Tasarım sürecinin eksiksiz tamamlanması için aşağıdaki adımların sırasıyla takip edilmesi gerekmektedir:

- 5'li (veya daha fazla kanallı) IR sensör diziliminin labirent zeminine göre siyah-beyaz (1-0) eşik kalibrasyonları donanımsal ve yazılımsal olarak yapılacaktır.
- Sensörlerin üretebileceği olası tüm lojik kombinasyonlar kağıt üzerine dökülecek ve labirentteki (T-kavşak, Sağ/Sol ayrımı, Çıkmaz Sokak vb.) fiziksel geometri karşılıkları belirlenecek.
- Keşif (Exploration) fazı için "Sol El Kuralı" veya "Sağ El Kuralı" hiyerarşisi belirlenerek, kavşaklarda robotun hangi yöne gideceğine karar veren asenkron kontrol bloğu (PID kontrolü ile birleştirilerek) koda eklenecek.
- Robotun yaptığı her bir net dönüş manevrasını (S: Sol, G: Sağ, İ: İleri, U: U-Dönüşü vb.) anlık olarak SRAM üzerindeki bir diziyi (array) karakter olarak kaydeden hafıza fonksiyonu yazılacaktır.
- Kağıt üzerinde vektörel olarak hesaplanan U-dönüşü sadeleştirme matrisi (Örn: $S+U+S = İ$) yazılımsal bir optimizasyon fonksiyonuna dönüştürülecek. Bu fonksiyon, robot her U-dönüşü yaptığında diziyi geriye dönük olarak tarayıp gereksiz yolları bellekten silecek.
- Keşif turu bitip hedefe ulaşıldığında (örneğin siyah bir kare alan algılandığında), robot motorlarını durdurup hafızasındaki son optimize edilmiş diziyi EEPROM'a veya geçici belleğe kalıcı olarak kilitleyecek.
- Optimizasyon (Hızlı Çözüm) fazında robot yeniden başlatıldığında, keşif algoritmalarını tamamen atlayarak sadece belleğindeki optimize edilmiş manevra listesini sırasıyla okuyup uygulayan ve hedefe duraksamadan giden nihai kod bloğu test edilecek.

Kaynaklar

- <https://www.alldatasheet.com/>
- <https://320volt.com/>
- <https://diyot.net/>
- <https://www.allaboutcircuits.com/>
- <https://www.electronics-tutorials.ws/>